Garbage Collection Annotated Bibliography

Sam Hiken

April 2026

"I love it because it's trash!"
— Oscar the Grouch

1 Introduction

Garbage collection is a form of automated memory management built into almost every modern programming language [10]. A garbage collection algorithm clears memory from the heap that is no longer accessible by the client program, freeing the programmer from having to manage memory allocation and de-allocation herself. Compared to languages with manual memory management, those using garbage collection are far less likely to suffer from bugs due to memory leaks and dangling pointers, and programs written in these languages are often easier to read and write than those that are not.

Formally, a garbage collection algorithm does the following. Let the *object graph* of a program at a given time step consist of nodes corresponding to objects stored in memory, together with an edge $a \to b$ whenever an object a contains a reference to an object b. The *root set* of the object graph consists of objects known to be in use by the program, such as the objects on the call stack. An object is said to be *live* if it is connected via a path from a root object. The purpose of a garbage collection algorithm is to reclaim the memory occupied by objects that are no longer live.

The main disadvantage of using a garbage collecting language is that it periodically requires the client program to halt so that the collection algorithm can run. Thus, the primary metric by which garbage collection schemes are measured is the typical *pause times* that they cause. The core thrust of garbage collection research over the last sixty-odd years, therefore, has been to design garbage collection schemes that minimize pause times under real-world conditions. In this annotated bibliography, we will explore a sampling of major techniques developed in this vast body of research.

Tracing algorithms Most garbage collectors fall into a broad category of algorithms called *tracing algorithms*. A tracing garbage collector is one that performs a tree-traversal from the root nodes of the object graph. Those locations in the heap not reached by the traversal are reclaimed by the collector.

There are three basic types of tracing collectors in widespread use. In increasing order of sophistication, they are (1) mark-sweep collectors, (2) mark-compact collectors, and (3) copying collectors [7, 10, 4]. We will discuss each of these collectors in detail in the course of this annotated bibliography.

Incremental garbage collection In their most naive implementations, the tracing algorithms mentioned above have a major disadvantage: periodically, they traverse the entire object graph, all in one fell swoop. This means that garbage collection is done in a small number of potentially very long pauses. In many applications in real-time systems, however, we wish to minimize not only the *total* pause time taken by the garbage collector throughout the client program's execution, but also the length of the longest *individual* pause. For these applications, it is better to use *incremental garbage collection*, which performs garbage collection over the course of short pauses dispersed throughout the execution of the program. The oldest incremental garbage collection technique is called *reference counting* [5]. A reference counting algorithm is one that maintains, for each object a in the heap, a counter for the number of references to the object. When a heap location's reference count goes from one to zero, it is reclaimed by the garbage collector. Reference counting algorithms come with various disadvantages we elaborate on in this annotated bibliography. A more widely used approach for incremental garbage collector is Baker's incremental tracing algorithm, which implements a form of copying collection [2].

Generational garbage collection The goal of garbage collection algorithm design is to minimize the length of the pauses taken by the collector. So far, we have been treating the object graph for a program as an arbitrary graph, and we have assumed that arbitrary objects go out of scope. These assumptions are both wrong, and computer scientists have designed heuristics to exploit typical structural properties that object graphs take on in practice. The most commonly used heuristic of this type is called *generational garbage collection*. Introduced by [6], generational garbage collection makes use of the following *generational hypothesis*: most objects die young. That is, most objects created by the program go out of scope relatively soon after being instantiated, and those that do not are usually part of some semi-permanent data structure unlikely to go out of scope in the near future. When the generational hypothesis holds, the garbage collector should therefore prioritize reclaiming memory that was allocated recently. Widely cited implementations of generational garbage collection include [9] and [1].

Parallel and concurrent garbage collection Two well-studied techniques for reducing pause times even further are parallel and concurrent garbage collection. A parallel garbage collector is a system that runs the client program and the collector in parallel (or, at least partially in parallel). Concurrent garbage collection programs, meanwhile, run the collector and the client program on separate overlapping threads. The major challenge these systems must overcome

is that the client program may alter the object graph as the collector traverses it. As a result, modern parallel and concurrent garbage collectors are highly complex. We look at a widely cited older and relatively simple parallel garbage collector by Boehm et al [3].

Open problems Garbage collectors in the twenty-first century have become highly complex, with cutting-edge concurrent collectors reducing average pause times to a fraction of a millisecond (see e.g [8]). As modern programs process ever more gigantic quantities of data, researchers have had to design garbage collection heuristics increasingly specialized to designated client programs. Major open problems in the area come from the need to fine-tune these algorithms even further.

2 Annotations to Bibliography

2.1 Wilson '94: survey paper [10]

Overview In "Uniprocessor Garbage Collection," Wilson surveys the history of uniprocessor (ie, not parallel or concurrent) garbage collection algorithms up until that point. Wilson identifies four basic types of garbage collection algorithms: (1) mark-sweep, (2) mark-compact, (3) copying collection, and (4) reference counting. As I describe (3) and (4) below, in this annotation I will focus on (1) and (2). A mark-sweep collection algorithm consists of the following:

1. *Mark phase*: in which the collector runs tree traversal through the object graph from the root nodes, and marks the locations that it reaches.

2. *Sweep phase*: in which the collector scans through the heap and reclaims the unmarked locations.

One of the big disadvantages of mark-sweep is that it leaves live objects scattered throughout the heap, which can make it difficult to allocate contiguous memory locations for very large objects later on.

To see one way of getting around this issue, we can take a look at another basic garbage collection algorithm, mark-compact. As in mark-sweep, the first phase of mark-compact runs tree-traversal from the root nodes of the object graph and marks the reachable memory locations. In phase two, however, mark-compact scans two pointers through the heap instead of one. One pointer scans through the heap. The other points to the earliest unoccupied location in the heap. When the first pointer encounters a marked object, the location is reclaimed and the object is moved to the location of the first pointer. In the end, all the live objects are moved into consecutive locations in memory.

How the author(s) evaluated their work As a survey paper written several decades after both of the above algorithms were introduced, [10] assesses them in the context of work done on garbage collection in the following decades.

It points out some of their limitations in the broad context of garbage collection algorithms: they are relatively slow, and do not implement the heuristics that are now commonplace in garbage collectors. However, they were important early work in the field that provide the basis for many algorithms that came later.

Limitations I discussed a major limitation of the mark-sweep algorithm above, namely that it can lead to live objects being dispersed across the heap in a way that could make future allocations of large objects difficult. While the mark-compact algorithm gets around this, its scanning step, like in mark-sweep, is computationally expensive, as it needs to scan the entire heap. While there have been variations on the mark-compact algorithm that avoid scanning the entire heap, they require complex recursive procedures in order to compact-ify the live objects. Cheney's algorithm, discussed below, addresses this issue.

2.2 Cheney '70: copying collection [4]

Overview In "A nonrecursive list compacting algorithm," Cheney gives the most famous version of a copying collection algorithm. The algorithm works as follows: the heap is split into two semi-spaces, the from-space and the to-space. Objects are stored in the from-space. Periodically, the algorithm runs tree traversal from the root objects, then copies the objects into the to-space. The from-space is then designated the new to-space, and vice-versa.

How the author(s) evaluated their work While Cheney implements their algorithm in assembly language, they do not design any empirical tests to assess the performance of the algorithm relative to garbage collection alternatives. Rather, their analysis of the algorithm is theoretical. The major advantage Cheney's algorithm has over the mark-compact algorithm described in the last paper is that it does not need to scan through the entire heap in order to assign the live objects contiguous locations in memory. Its runtime is proportional to the *number of live objects*, rather than the size of the entire heap. This is because, after the live objects are copied over to the to-space, the program can assume that all the memory in the from-space has been reclaimed. Therefore, the pause time is proportional to the *number of live objects*, rather than to the size of the entire heap. While there are implementations of mark-compact algorithms that avoid this issue, they rely on complex recursive procedures that, at least in Cheney's time, were not practical.

Limitations As we will see, later research made substantial advancements in copying collection algorithms. One major limitation of Cheney's algorithm and other early garbage collection schemes like it is that the collection is performed in occasional relatively expensive computations, in which the live objects are all traversed in one go. This becomes infeasible in real-time computing applications, where the important metric is not the *total pause time* taken during the

program's execution, but rather the *worst case* time taken during any pause. The alternative to the approaches we have see so far is *incremental garbage collection*, in which garbage collection is performed gradually over the course of the client program's execution.

2.3 Collins '60: reference counting [5]

Overview [Note: I made a typo in my presentation, and accidently attributed this paper to "Cohen" rather than "Collins".] In "A method for overlapping and erasure of lists," Collins introduces reference counting, the fourth basic garbage collection algorithm. In reference counting, each object has an associated *reference count*, which tracks how many other objects reference it. Whenever a new object a is created with a reference to b, b's counter is incremented. When a later goes out of scope, b's counter is decremented. When an object's counter goes from 1 to 0, it is no longer reachable from the root objects and can be reclaimed by the collector. A major advantage of reference counting is that it is *incremental*, that is, the length of each pause is proportional to the number of objects that go out of scope at the corresponding time-step of the program. This is in contrast to the other three basic algorithms described so far, whose maximum pause times are of length proportional (at least) to the current number of live objects. Incremental collection is critical for garbage collection in real-time systems, when maximum pause times must be kept low.

How the author(s) evaluated their work As this is one of the very earliest papers on garbage collection, Collins does not perform detailed experiments comparing his work to other garbage collectors. Rather, he is mainly focused on contrasting his algorithm with the original mark-sweep algorithm by [7]. As he points out, that algorithm requires scanning the entire heap in order to reclaim memory. After providing pseudocode for his algorithm, he correctly notes that it avoids this issue encountered by [7].

Limitations The most important disadvantage of reference counting algorithms is that they fail to reclaim cycles in the object graph that are unreachable from the client program's roots. This is because each object in a cycle will have reference count at least one, even if the cycle is isolated from the rest of the object graph. In order to prevent these cycles from accumulating in the heap during the program's execution, reference counting in practice must often be implemented in conjunction with other garbage collection algorithms.

2.4 Baker '78: incremental tracing collection [2]

Overview In this paper, Baker designs an algorithm that captures the advantages both of tracing algorithms (like mark-sweep, mark-compact, and copying collection), that is also incremental. The algorithm is a variation on copying collection. Unlike Cheney's stop-and-copy algorithm, Baker's algorithm does not copy over the entire object graph from the from-space to the to-space all in

one go. Rather, the garbage collection is broken into some number of epochs. During each epoch, the client program is run. Whenever a location in the from-space is accessed by the program, the object in that location is copied over to the to-space, and the program is allowed to proceed. This allows the copy collection algorithm to be made incremental.

How the author(s) evaluated their work Baker's original paper does not provide an implementation of the algorithm that may be compared to other garbage collectors. Rather, it contains a highly detailed theoretical accounting of the algorithm's expected performance in terms of runtime and space constraints.

Limitations As Baker does not implement the algorithm, he leaves open the task of actually assessing its performance in real-world circumstances. He also leaves as future work the task of studying how the algorithm may interact with garbage collection heuristic, like those which optimize the order in which memory locations are explored by the collector. We see an example of such a heuristic below.

2.5 Appel '89: generational collection [1]

Overview Perhaps the most widely used heuristic in garbage collection is *generational garbage collection*, which makes use of the *generational hypothesis* that "most objects die young." In other words, most objects created by the program go out of scope relatively soon after being instantiated, and those that do not are usually part of some semi-permanent data structure that is unlikely to have gone out of scope since the previous garbage collection cycle. Appel provides an implementation of garbage collection that provides faster allocation time than earlier implementations. The algorithm roughly works as follows. The heap is divided into "areas" $A_1, ..., A_n$, where each area contains objects that were created during the same time interval. The algorithm uses the following observation: in most programs, most edges in the object graph go from newer objects to older objects. Therefore, if the algorithm performs a *partial* tree traversal from the roots, halting once it reaches objects that are of age i, it will have discovered most objects of age at most i, which correspond to those contained in heap areas $A_1, .., A_i$. By the generational hypothesis, most dead objects are of age at most i, for some reasonable choice of i. Thus, generational collection algorithms can get away with performing a tracing algorithm that only traverses part of the object graph, reducing pause times over non-generational algorithms considerably.

How the author(s) evaluated their work Primarily, Appel was concerned with reducing *allocation times* for this algorithm relative to other generational collectors. This is because with generational collection, the process of allocating memory can often be *more expensive* than reclaiming it, at least in programs

that perform relatively few allocations. He provides a detailed theoretical accounting to demonstrate the speed-ups. He also provides an implementation of the algorithm in C, and demonstrates that it allocates memory "like crazy," (his words) without creating too much overhead for the collector.

Limitations Appel's implementation of generational collection has a collector that is modestly slower than the state-of-the-art for the time, so in programs where the pause time bottleneck comes from the cost of collection, rather than the cost of allocation, Appel's algorithm may not be preferable. Moreover, garbage collectors have come a long way since the 1980's. Most cutting-edge collectors today use some form of parallelism or concurrency to speed up the computation. We will see an example of such a collector below.

2.6 Boehm et al, '91: parallel collection [3]

Overview Boehm et al. provide a general purpose technique for turning a sequential tracing collection algorithm into a mostly parallel algorithm. A cycle of the mostly parallel algorithm proceeds as follows. The client program and tracing algorithm are run in parallel. Meanwhile, a body of *virtual memory* is maintained outside of the heap. As the client program proceeds, pointers are stored in virtual memory to locations in the heap accessed by the client program during the current cycle. After the tracing algorithm terminates, the client program is paused, and a tree traversal is run from the heap locations stored in virtual memory. Thus, most of the garbage collection is run in parallel with the client program.

How the author(s) evaluated their work The authors run various experiments comparing their technique to non-parallel garbage collection programs, and find that their parallel generational collector has substantially shorter pause times than a non-parallel generational collector they compare it to.

Limitations One limitation discussed in the paper is the fact that the techniques they give do not provide a way of parallelizing *copying collectors*, which, of the four basic garbage collection algorithms (mark-sweep, mark-compact, copying collection, and reference counting), is the most widely implemented. Another more general disadvantage to parallel garbage collection is that it requires coordination across multiple cores. For this reason, concurrent garbage collection is more common in modern applications.

3 Bibliography

References

[1] Andrew W. Appel. "Simple generational garbage collection and fast allocation". In: *Software: Practice and Experience* 19.2 (1989), pp. 171–183.

DOI: https://doi.org/10.1002/spe.4380190206. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.4380190206. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.4380190206.

[2] Henry G. Baker. "List processing in real time on a serial computer". In: *Commun. ACM* 21.4 (Apr. 1978), pp. 280–294. ISSN: 0001-0782. DOI: 10.1145/359460.359470. URL: https://doi.org/10.1145/359460.359470.

[3] Hans-J. Boehm, Alan J. Demers, and Scott Shenker. "Mostly parallel garbage collection". In: *Proceedings of the ACM SIGPLAN 1991 Conference on Programming Language Design and Implementation*. PLDI '91. Toronto, Ontario, Canada: Association for Computing Machinery, 1991, pp. 157–164. ISBN: 0897914287. DOI: 10.1145/113445.113459. URL: https://doi.org/10.1145/113445.113459.

[4] C. J. Cheney. "A nonrecursive list compacting algorithm". In: *Commun. ACM* 13.11 (Nov. 1970), pp. 677–678. ISSN: 0001-0782. DOI: 10.1145/362790.362798. URL: https://doi.org/10.1145/362790.362798.

[5] George E. Collins. "A method for overlapping and erasure of lists". In: *Commun. ACM* 3.12 (Dec. 1960), pp. 655–657. ISSN: 0001-0782. DOI: 10.1145/367487.367501. URL: https://doi.org/10.1145/367487.367501.

[6] Henry Lieberman and Carl Hewitt. "A real-time garbage collector based on the lifetimes of objects". In: *Commun. ACM* 26.6 (June 1983), pp. 419–429. ISSN: 0001-0782. DOI: 10.1145/358141.358147. URL: https://doi.org/10.1145/358141.358147.

[7] John McCarthy. "Recursive functions of symbolic expressions and their computation by machine, Part I". In: *Commun. ACM* 3.4 (Apr. 1960), pp. 184–195. ISSN: 0001-0782. DOI: 10.1145/367177.367199. URL: https://doi.org/10.1145/367177.367199.

[8] Katsuhiro Ueno and Atsushi Ohori. "A fully concurrent garbage collector for functional programs on multicore processors". In: *SIGPLAN Not.* 51.9 (Sept. 2016), pp. 421–433. ISSN: 0362-1340. DOI: 10.1145/3022670.2951944. URL: https://doi.org/10.1145/3022670.2951944.

[9] David Ungar. "Generation Scavenging: A non-disruptive high performance storage reclamation algorithm". In: *Proceedings of the First ACM SIGSOFT/SIGPLAN Software Engineering Symposium on Practical Software Development Environments*. SDE 1. New York, NY, USA: Association for Computing Machinery, 1984, pp. 157–167. ISBN: 0897911318. DOI: 10.1145/800020.808261. URL: https://doi.org/10.1145/800020.808261.

[10] Paul R. Wilson. "Uniprocessor garbage collection techniques". In: *Memory Management*. Ed. by Yves Bekkers and Jacques Cohen. Berlin, Heidelberg: Springer Berlin Heidelberg, 1992, pp. 1–42. ISBN: 978-3-540-47315-2.